

Discovery Executive Summary

Project: dicoverry-123 · Generated: 27/07/2026, 10:43:28

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 4 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	42 / 100 — Moderate
3	Frontend Modernization Analysis	HIGH RISK	—
4	Performance & Sustainability Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service/Repository layers (H2/H3), Domain Boundary Violations (H8), Shared DB Coupling (H9), and Dual-API duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a modular monolith (Discovery IVR + Connect TFN) with a thin Laravel 12 API, a React 19 SPA, and a parallel Express `dev-api` that re-implements the same workflows. Controllers are short by LOC but fat by responsibility: Eloquent and KPI math live in HTTP handlers, Modules folders are documentation stubs only, and there are zero repository classes. The dominant risk is change amplification from duplicated business logic across Laravel, Express, and the SPA, plus shared Mongo collections and cross-domain dashboard/legacy reports that erase bounded-context ownership. Frontend has a usable `api/client` and React Query, but legacy class/widget patterns and hard-coded endpoint strings in pages still couple UI to backend shapes.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC (Laravel avg); Express <code>server.js</code> 224 LOC	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 handler methods with direct model/store access	HIGH
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	35+ Eloquent/store/Mongo access points; 0 repositories	HIGH
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (<code>LegacyDataMapper</code> , shared <code>buildTree</code> helpers)	MODERATE
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	~38% of queries outside controllers	HIGH
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest <code>server.js</code> 224 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8 cross-domain access points	HIGH
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	37.5% (3/8 stores shared via Mongo <code>module</code>)	HIGH
H10	Dual API Duplication (additional)	Duplicated workflows across Laravel ↔ Express	0	1–3	>3	5+ workflows duplicated	HIGH
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components with hard-coded paths	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest ConnectPage 208 LOC)	GOOD

F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels; small Zustand UI store	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (<code>LegacyMonitorPoller</code> , <code>LegacyDashboardWidget</code>)	MODE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Extract reachability/tree/KPI logic from <code>ConnectController</code> , <code>LegacyReportController</code> , and Express <code>server.js</code> into Application Services	MODERATE	HIGH
H2 Missing Service Layer	Create <code>DiscoveryJobService</code> , <code>ConnectMonitorService</code> , <code>DashboardKpiService</code> ; stop Eloquent/store use in handlers	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces for Eloquent models; bind in <code>AppServiceProvider</code> ; keep Mongo behind repository APIs	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> <code>extract()</code> with DTOs; single <code>IvrTreeBuilder</code> domain service	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Enforce repository-only persistence; remove Eloquent from <code>Http/Controllers</code> and Mongo from Express routes	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Populate real <code>Modules/Discovery</code> & <code>Modules/Connect</code> ; move Reporting behind published DTOs/ACL	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Split Mongo collections per BC; document MariaDB ownership; ban cross-module writes	HIGH RISK	CRITICAL
H10 Dual API Duplication	Consolidate on Laravel as single API; proxy or delete Express <code>dev-api</code> duplication	HIGH RISK	CRITICAL
F5 Legacy Component Patterns	Convert <code>LegacyMonitorPoller</code> to hooks; add Error Boundaries; remove unused legacy widget	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers become thin HTTP adapters; Application/Domain Services own workflows and are reusable from jobs/CLI.
- Repository interfaces enable testing without MariaDB/Mongo and isolate schema churn to one layer.
- Real bounded contexts (Discovery, Connect, Reporting) stop silent cross-module coupling and support future extraction.
- Eliminating the dual Laravel/Express stack removes the largest source of divergent business rules.
- Frontend domain API modules plus purged legacy components keep the SPA aligned with a single backend contract.

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by H3 Large Functions (ConnectPage 201 LOC), H5 Duplicate Code (~12.3%), and H9 unsafe extract() (3 call sites).

EXECUTIVE SUMMARY

Klearcom is a small multi-layer platform (Laravel API, React SPA, Node `dev-api`) with no runnable complexity tooling configured (phpstan is listed in Composer but unused; no ESLint complexity rule), so metrics were measured by manual branch/LOC inspection across **39 application source files** (backend 18, frontend 15, `dev-api` 6). The worst findings are a **201-LOC** `ConnectPage` component (ESLint-style CC ≈ 20), ~**8.8–12.3%** duplicated business/UI logic (IVR `buildTree` ×3, reachability math ×4, dual Laravel/`dev-api` realtime runners, near-clone Discovery/Connect pages), and **3** unsafe `extract()` call sites in legacy PHP. Git history is short (3 commits, single author `ksabai-gl`); churn and ownership look healthy, but they do not offset the structural duplication and size risks. Overall rating is **High Risk**, driven by large functions, general duplication, and `extract()`.

2.1 Benchmark Ratings Summary

Layers covered: **Backend** (Laravel `backend/app` + routes/tests = 18 files) · **Frontend** (React `frontend/src` = 15 files) · **Dev API** (Node `dev-api/src` = 6 files). Tooling: manual LOC/branch counts (no ESLint complexity, no radon/gocyclo, phpstan present in `composer.json` but no config/run).

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	20 (<code>ConnectPage</code> , ESLint-style) · FE DiscoveryPage 18 · BE-JS <code>runConnectTest</code> 15 · BE-PHP <code>runConnectTest</code> 10	MODERATE
H2	Large Classes	Largest class/file LOC	<300	300–1000	>1000	224 (<code>dev-api/src/server.js</code>); next FE <code>ConnectPage.tsx</code> 208	GOOD
H3	Large Functions	Largest function LOC	<50	50–200	>200	201 (<code>ConnectPage</code>); FE DiscoveryPage 154 · BE max ~64 (<code>runConnectTest</code>)	HIGH RISK
H4	Business Logic	Duplicated business	<5%	5–10%	>10%	~8.8% (<code>buildTree</code> ×3, reachability×4, dual	MODERATE

	Duplication	logic %				realtime, KPI×2)	
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~12.3% (H4 + Discovery/Connect page UI clone)	HIGH RISK
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	~2 (June 2026; top files touched twice)	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	1 (<code>ConnectController</code> / <code>server.js</code> / <code>App.tsx</code> in fix commits)	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (<code>ksabai-gl</code>)	GOOD
H9	Unsafe <code>extract()</code> / dynamic vars (additional)	<code>extract()</code> call sites (Good 0 · Moderate 1–2 · High Risk ≥3)	0	1–2	≥3	3	HIGH RISK
H10	Missing lifecycle cleanup (additional)	Uncleared interval/SSE components (Good 0 · Moderate 1 · High Risk ≥2)	0	1	≥2	1 (<code>LegacyMonitorPoller</code>)	MODERATE

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	66	16.5
Code Churn	25%	18	4.5
Defect Density	20%	15	3.0
Class/Function Size	15%	68	10.2
Business Logic Duplication	10%	72	7.2
Developer Ownership Risk	5%	8	0.4
Hotspot Score	100%		42 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
H1 High Cyclomatic Complexity	Split <code>ConnectPage</code> conditionals into subcomponents; extract shared reachability helpers; enable ESLint <code>complexity</code>	MODERATE	MEDIUM

H3 Large Functions	Break <code>ConnectPage</code> (201 LOC) and shrink <code>DiscoveryPage</code> (154 LOC) via hooks + presentational children	HIGH RISK	CRITICAL
H4 Business Logic Duplication	Consolidate <code>buildTree</code> and reachability math into domain services used by Laravel and <code>dev-api</code>	MODERATE	HIGH
H5 Duplicate Code (general)	Extract shared page layout / transcript / query patterns; add duplication detection in CI	HIGH RISK	CRITICAL
H9 Unsafe <code>extract()</code>	Replace 3 <code>extract()</code> sites with explicit validated arrays / DTOs; gate in CI	HIGH RISK	CRITICAL
H10 Missing lifecycle cleanup	Add unmount cleanup to <code>LegacyMonitorPoller</code> or migrate to React Query	MODERATE	MEDIUM

2.6 Expected Outcomes

- Lower defect rate on Connect/Discovery changes by testing smaller components and a single reachability policy.
- Safer refactors when IVR tree or alert thresholds change — one service update instead of three+ copies.
- Easier code review once page components stay under ~80 LOC and ternaries are flattened.
- Clearer ownership of domain rules (`IvrTreeBuilder` , `ReachabilityPolicy`) versus UI shells.
- Elimination of `extract()` and interval-leak debt reduces security and runtime footguns in legacy paths.

3. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND DISCOVERY

High Risk

Driven by H1 (page duplication), H6 (empty feature modules), H8 (30 inline styles), H9 (0% route guards), H15 (no browserslist/polyfills), and H18 (no Error Boundaries).

EXECUTIVE SUMMARY

The Klearcom frontend is a modern React 19.2 / TypeScript / Vite 6 SPA with a solid API client, TanStack Query caching on primary pages, CSS design tokens, and TypeScript `strict: true`. The largest gaps are structural: `modules/Discovery` and `modules/Connect` are empty shells (pages own all feature UI), `DiscoveryPage` and `ConnectPage` are near-duplicate layouts, and there is no authentication or route guarding on operational screens. Inline `style={{...}}` usage is widespread (30 occurrences), ESLint and browserslist are absent, and two High CVEs ship via `react-router` 7.18.x. Dead legacy artifacts (`LegacyMonitorPoller` class component, `LegacyDashboardWidget`) remain in-tree with interval leaks and uncaught throws, and no Error Boundary or accessibility roles were found.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	22% (2/9 near-duplicate pages)	HIGH RISK
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	89% (8/9 functional)	MODERATE
H3	Massive Components	Largest component LOC	<200	200–500	>500	222 LOC (<code>ConnectPage.tsx</code>)	MODERATE
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	22% (2/9 use Zustand)	GOOD
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	2 levels	GOOD
H6	Weak Frontend Architecture	Feature modules with clean boundaries %	>80%	50–80%	<50%	0% (modules are AGENTS.md only)	HIGH RISK
H7	Missing Component Inventory	Shared component % of total	>30%	15–30%	<15%	44% (4/9 in <code>components/</code>)	GOOD
H8	No Design System	Inline-style / magic-value occurrences	0–5	6–20	>20	30 <code>style={{...}}</code>	HIGH RISK
H9	Routing Structure Weakness	Protected routes with guards %	100%	80–99%	<80%	0% (0/3 routes guarded)	HIGH RISK
H10	No API Integration Layer	API calls in service layer %	>90%	70–90%	<70%	100% via <code>api/client.ts</code>	GOOD
H11	Poor Data Caching	Data-fetching points with caching %	>70%	40–70%	<40%	67% (8/12 use React Query)	MODERATE
H12	Weak Frontend Auth	Token storage + routes guarded	httpOnly + 100%	One gap	Both gaps	No auth; 0% guarded	MODERATE

H13	Frontend Security Vulnerabilities	XSS-risk + hardcoded secrets count	0 each	1–3 total	>3 total	1 (CDN font CSS, no SRI)	MODERATE
H14	Frontend Performance Gaps	Initial JS bundle size (gzipped)	<250KB	250–500KB	>500KB	151 KB gzip (single chunk)	GOOD
H15	Browser Compatibility Gaps	Browserslist + polyfills configured	Both present	One missing	Both missing	Both missing	HIGH RISK
H16	Frontend Code Quality	ESLint in CI + TypeScript strict	Both Yes	One Yes	Both No	Strict Yes; ESLint No	MODERATE
H17	Technical Debt & Dependencies	Critical/High CVEs found	0	1–3	>3	2 High (<code>react-router</code>)	MODERATE
H18	Missing Error Boundaries (additional)	Route trees wrapped by Error Boundary (target 100%)	100%	1–99%	0%	0%	HIGH RISK
H19	Missing Accessibility Semantics (additional)	Interactive UI with <code>aria-*</code> / <code>role</code> usage (target present)	Present	Sparse	None observed	0 matches	HIGH RISK

3.9 Actions Required

Hotspot	Action	Rating	Priority
H1 UI Component Duplication	Extract shared form/table/transcript components; stop copying Discovery ↔ Connect page shells	HIGH RISK	CRITICAL
H2 Legacy Class-Based Components	Remove or rewrite <code>LegacyMonitorPoller</code> / imperative widget with hooks + React Query	MODERATE	MEDIUM
H3 Massive Components	Split <code>ConnectPage</code> / <code>DiscoveryPage</code> into feature subcomponents under modules	MODERATE	MEDIUM
H6 Weak Frontend Architecture	Move real feature code into <code>modules/{Discovery,Connect}</code> with public exports and import rules	HIGH RISK	CRITICAL
H8 No Design System	Replace 30 inline <code>style={{...}}</code> usages with CSS utility classes on top of existing tokens	HIGH RISK	HIGH
H9 Routing Structure Weakness	Add auth-aware route config with <code>React.lazy</code> for Discovery/Connect	HIGH RISK	CRITICAL

H11 Poor Data Caching	Delete uncached legacy pollers; keep all fetches on React Query	MODERATE	MEDIUM
H12 Weak Frontend Auth	Implement httpOnly session auth and centralized <code>RequireAuth</code> guards	MODERATE	HIGH
H13 Frontend Security Vulnerabilities	Self-host fonts or add SRI integrity on Google Fonts CSS	MODERATE	MEDIUM
H15 Browser Compatibility Gaps	Add <code>.browserslistrc</code> and EventSource feature detection UX	HIGH RISK	HIGH
H16 Frontend Code Quality	Add ESLint + react-hooks/jsx-a11y and enforce in CI	MODERATE	MEDIUM
H17 Technical Debt & Dependencies	Upgrade <code>react-router-dom</code> past GHSA-qwww-vcr4-c8h2; remove dead <code>Legacy*</code> files	MODERATE	HIGH
H18 Missing Error Boundaries	Wrap routes in an Error Boundary with recoverable fallback UI	HIGH RISK	CRITICAL
H19 Missing Accessibility Semantics	Add ARIA/keyboard support for nav, tables, and live feed; enable jsx-a11y	HIGH RISK	HIGH

3.10 Expected Outcomes

- Shared form/table/transcript components eliminate Discovery ↔ Connect drift and cut duplicate fix cost.
- Real `modules/{Discovery,Connect}` packages make feature ownership and AGENTS.md guidance accurate.
- httpOnly auth plus route guards prevent anonymous operation of IVR/TFN tooling.
- Error Boundaries convert white-screen failures into recoverable UI.
- Token-backed CSS utilities remove inline-style sprawl and speed visual changes.
- ESLint + jsx-a11y in CI catch hooks and accessibility regressions before merge.
- Patching `react-router` clears the two High CVEs from `npm audit`.
- Browserslist + SSE detection make realtime-test support explicit for target browsers.

4. Performance & Sustainability Analysis

OVERALL CODEBASE RATING — PERFORMANCE & SUSTAINABILITY

High Risk

Driven by P2 Database, P3 API, P4 Memory, P6 Concurrency, P8 Resource Utilization, P9 Network, and P12 Sustainability.

EXECUTIVE SUMMARY

Klearcom's dual runtime (Laravel + Node `dev-api`) is a small voice-observability platform with clear efficiency debt concentrated in data access, long-lived streaming, and always-on Docker resources. The highest-risk patterns are an N+1

query loop in `LegacyReportController`, unbounded MariaDB list loads, SSE endpoints that re-read full Mongo event histories every 500 ms while holding request workers with `usleep`, and a five-service Compose stack with no CPU/memory limits that ships the frontend via `npm run dev`. Frontend polling (`refreshInterval`, `LegacyMonitorPoller` every 3 s, `MongoStatus` every 15 s) amplifies traffic while nginx lacks gzip. CI installs Composer/npm and pecl MongoDB with no dependency caching. Overall rating is **High Risk**, driven by database, API latency, memory retention, concurrency, network chatter, resource waste, and sustainability posture.

8.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	3	MODERATE
P2	Database Performance	Slow-query / N+1 sites	0	1–5	>5	6	HIGH RISK
P3	API Performance	Response-latency hotspots	0	1–5	>5	6	HIGH RISK
P4	Memory Efficiency	High-memory sites	0	1–3	>3	4	HIGH RISK
P5	CPU Efficiency	CPU-intensive operations	0	1–5	>5	3	MODERATE
P6	Concurrency	Blocking / sequential sites	0	1–5	>5	6	HIGH RISK
P7	Caching	Missed caching opportunities	0	1–5	>5	5	MODERATE
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	4	HIGH RISK
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	6	HIGH RISK
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	partial	MODERATE
P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	1	MODERATE
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	wasteful	HIGH RISK
P13	Missing client timeouts (additional)	Fetch/HTTP calls without timeout	0	1–2	>2	1	MODERATE

P14	Uncleared polling intervals (additional)	Intervals without unmount cleanup	0	1	≥ 2	1	MODERATE
-----	--	-----------------------------------	---	---	----------	---	-----------------

8.5 Actions Required

Hotspot	Action	Rating	Priority
P2 Database Performance	Eliminate N+1 in <code>LegacyReportController</code> ; paginate list endpoints; add indexes on status/country/checked_at; cursor-limit Mongo event reads	HIGH RISK	CRITICAL
P3 API Performance	Move simulated tests to queue workers; replace SSE Mongo re-poll with pub/sub; stop parallel REST refetch during live runs	HIGH RISK	CRITICAL
P6 Concurrency	Stop holding PHP-FPM/Node with <code>usleep</code> /interval polls; introduce worker pool and push-based streams	HIGH RISK	CRITICAL
P4 Memory Efficiency	Cap <code>getTestEvents</code> and in-memory store growth; paginate Eloquent collections	HIGH RISK	HIGH
P8 Resource Utilization	Add Compose resource limits; ship static frontend image; disable APP_DEBUG outside local; unpublish DB ports in non-dev	HIGH RISK	HIGH
P9 Network Efficiency	Prefer SSE-only live updates; remove legacy 3 s/10 s pollers; enable nginx gzip/brotli	HIGH RISK	HIGH
P12 Sustainability	Right-size always-on stack and cut wasteful poll/N+1 paths; plan autoscaled workers for AWS	HIGH RISK	HIGH
P1 Algorithm Efficiency	Replace recursive full-scan <code>buildTree</code> with parent_id hash grouping (O(n)) shared across Laravel and <code>dev-api</code>	MODERATE	MEDIUM
P5 CPU Efficiency	Ping-only health checks; avoid full collection counts on the 15 s status path	MODERATE	MEDIUM
P7 Caching	Cache dashboard KPIs and IVR trees; short-circuit repeated health work	MODERATE	MEDIUM
P10 Build Efficiency	Cache Composer/npm and avoid pecl rebuild every CI run	MODERATE	MEDIUM
P11 Logging Efficiency	Default <code>APP_DEBUG=false</code> outside local Compose profile	MODERATE	MEDIUM
P13 Missing client timeouts	Add <code>AbortSignal.timeout</code> to <code>frontend/src/api/client.ts</code>	MODERATE	MEDIUM
P14 Uncleared polling intervals	Clear <code>LegacyMonitorPoller</code> interval on unmount	MODERATE	MEDIUM

8.6 Expected Outcomes

- Batched/indexed MariaDB access and paginated lists remove N+1 and unbounded payload latency as monitors and jobs scale.
- Queue-backed tests plus pub/sub SSE free PHP-FPM/Node workers, raising concurrent Discovery/Connect throughput.
- O(n) tree builds, capped Mongo reads, and server-side KPI/tree caches cut CPU, memory, and repeated query cost.

- gzip, SSE-only live updates, and removal of leaked/legacy pollers reduce chatty traffic and bandwidth.
- Right-sized Compose (static FE, debug off, limits) plus CI dependency caching lower cloud cost, idle energy use, and carbon footprint.